Closed-Loop Text-to-CAD Generation and Metrological Validation of Functional Engineering Components via LLM-Driven Python Scripting, 3D Printing, and Optical Scanning

Renzo Alberto Taco Coayla
Doctor en Administración
rentaco@upt.pe
https://orcid.org/0000-0003-0014-7216
Universidad Privada de Tacna
Teléfono: 948813542

Martha Judith Paredes Vignola
Doctora en Educación con Mención en Gestión Educativa
majparedes@upt.pe
https://orcid.org/0000-0002-5046-0908
Universidad Privada de Tacna
Teléfono: 952341059

Elard Ricardo Rodriguez Marca
Magister en Docencia
elrrodriguez@upt.pe
https://orcid.org/0000-0002-1220-1972
Universidad Privada de Tacna
Teléfono: 977138669

Fabrizio Salvador Elias Perez Peralta
Estudiante Ingeniería de Sistemas
fabperez@upt.pe
https://orcid.org/0009-0005-3906-7856
Universidad Privada de Tacna
Teléfono: 964501290

Abstract

1. Introduction

In the transition toward Industry 4.0, generative artificial intelligence has begun to democratize computer-aided design (CAD), drastically lowering entry barriers for three-dimensional modeling and rapid prototyping (Daareyni et al., 2025). Traditionally, creating mechanical components and parametric geometries in industrial CAD software demands a substantial expenditure of time alongside rigorous technical expertise in navigating complex user interfaces (Khan et al., 2024). In this landscape, Large Language Models (LLMs) and Vision-Language Models (VLMs) emerge not merely as passive conversational agents, but as symbolic and spatial

reasoning systems capable of interpreting human natural language instructions and directly interacting with engineering software engines (Alrashedy et al., 2025; Zhang et al., 2025). This integration of autonomous agents into design workflows promises to accelerate conceptual phases and streamline product life-cycle management through the automated execution of complex design tasks (Sun et al., 2025).

Unlike initial Text-to-3D approaches that produced non-editable meshes—such as static STL files or point clouds lacking parametric representation (Khan et al., 2024)—recent investigations have harnessed LLM capabilities to synthesize parametric construction code (e.g., CadQuery or Python-OpenCASCADE), enabling engineers to retain design history and boundary representation (B-Rep) editability (Sun et al., 2025; Xie & Ju, 2025). Generating executable Python code directly provides a high-level abstraction for defining geometric primitives, extrusions, and Boolean operations without reliance on proprietary software suites (Guan et al., 2025). Furthermore, state-of-the-art frameworks demonstrate that coupling LLMs with programmable CAD libraries facilitates the seamless incorporation of adjustable parameters and topological constraints essential for downstream manufacturing (Li et al., 2026; Niu et al., 2026).

Despite these algorithmic advancements, a critical disconnect persists in the literature: validation stops at the virtual environment (Daareyni et al., 2025). Evaluating an AI-generated CAD model solely through its compile rate, parsing rate, or digital Chamfer distance over synthetic point clouds ignores the topological and thermal complexities inherent to additive manufacturing (Li et al., 2024; Sun et al., 2025). During fused deposition modeling (FDM) or vat photopolymerization (SLA/DLP) processes, physical parts experience intrinsic phenomena such as thermal contraction, volumetric resin shrinkage, warping, and interfacial anisotropy that distort mechanical tolerances and dimensional fidelity (Ling et al., 2022; Patchanee et al., 2025). Consequently, high accuracy on a computer screen in no way guarantees the assembly viability or functional performance of the 3D-printed component (Daghrery et al., 2025; Etemad-Shahidi et al., 2020).

To bridge this cyber-physical divide, this study proposes a closed-loop experimental framework that transitions seamlessly from natural language intent to physical metrological validation. The system integrates a local LLM agent optimized for CadQuery Python script synthesis, supported by a real-time execution feedback loop for automated syntactic debugging (Alrashedy et al.,

2025). Following the synthesis of the digital B-Rep solid and its subsequent fabrication via FDM and SLA 3D printing, the physical part is digitized using a high-resolution 3D optical scanner (Patchanee et al., 2025). The resulting dense point cloud undergoes rigid spatial alignment using the Iterative Closest Point (ICP) algorithm against the nominal CAD model, enabling the extraction of physical metrological ground truth and the precise quantification of 3D spatial deviations (Azpiazu-Flores et al., 2023; Daghrery et al., 2025).

The primary contributions of this research are fourfold:

- **Propose** a cyber-physical closed-loop framework (Closed-Loop Text-to-Physical CAD) that unifies local LLM-driven parametric code generation with additive manufacturing and high-precision 3D optical metrology.
- **Implement** a Python (CadQuery) code generation agent equipped with an execution sandbox and a syntactic self-correction debugger (execution feedback loop) to maximize compile rate and topological validity in B-Rep solids.
- **Quantify** systematically the dimensional gap between virtual representations and manufactured components, contrasting digital synthetic metrics (compile rate, IoU, virtual Chamfer distance) against physical metrological ground truth (physical Hausdorff distance, nominal GD&T dimensional error, and volumetric deviation).
- **Validate** experimentally the manufacturing viability and geometric fidelity across functional mechanical components of varying structural complexity (gears, protective enclosures, and adaptive prosthetics), establishing a reproducible metrological standard for certifying generative engineering tools.

2. Related Work

To contextualize the contributions of this research within the state of the art, a systematic literature review encompassing 128 peer-reviewed publications (2022–2026) was conducted. The analysis evaluates recent advancements along three critical dimensions: parametric code generation, automated syntactic execution feedback, and physical metrological validation in additive manufacturing.

2.1. Large Language Models in Parametric CAD Generation

The evolution of 3D generative artificial intelligence has undergone a fundamental paradigm shift, moving away from early implicit neural representations such as voxels, point clouds, and non-editable meshes (Khan et al., 2024) toward explicit, parametric computer-aided design (CAD) representations (Daareyni et al., 2025; Zhang et al., 2025). Initial Text-to-3D models generated raw Stereolithography (STL) files or triangular boundary representations that lacked design history, rendering them unsuitable for downstream mechanical engineering applications where feature editing and tolerance adjustment are mandatory (Khan et al., 2024). To overcome these constraints, recent literature has prioritized the generation of domain-specific programming code that constructs Boundary Representation (B-Rep) solids (Guan et al., 2025; Xie & Ju, 2025).

Quantitative analysis of the extracted literature reveals that among generative CAD frameworks published between 2024 and 2026, 47.5% adopt CadQuery/Python as their primary modeling engine, followed by OpenCASCADE/FreeCAD API integrations (35.6%), Constructive Solid Geometry (CSG) via OpenSCAD (8.5%), and multimodal JSON/NURBS representations (5.1%) (Li et al., 2026; Sun et al., 2025). CadQuery has emerged as the preferred standard because its Python-native syntax leverages the massive code pre-training of modern Large Language Models (LLMs), enabling precise construction of complex B-Rep primitives, sketch-and-extrude operations, fillets, and chamfers through structured object-oriented scripting (Guan et al., 2025; Xie & Ju, 2025). Prominent frameworks such as Text2CAD (Khan et al., 2024), CAD-Coder (Guan et al., 2025), and Text-to-CadQuery (Xie & Ju, 2025) have demonstrated high syntactic fluency in generating sequential CAD commands from expert text prompts.

However, a critical limitation persists across these state-of-the-art generative pipelines: they operate under the assumption that generated code can be reliably compiled in a single forward pass without spatial reasoning or constraint validation (Niu et al., 2026). Frameworks like GenCAD (Li et al., 2024) and CAD-GPT (Sun et al., 2025) achieve competitive Intersection over Union (IoU) scores on synthetic benchmarks (e.g., DeepCAD or Fusion360 datasets), yet they struggle when prompts demand exact engineering constraints, such as pitch diameters in gear teeth or interlocking enclosure tolerances. Without domain-specific constraint grounding, pure text-to-code synthesis frequently produces self-intersecting geometries or invalid B-Rep topological boundaries (Li et al., 2026).

To address these representation gaps, **Agente-Ciel** establishes a parametric CadQuery code generation framework driven by structured Chain-of-Thought (CoT) prompting and local open-source LLMs (Qwen2.5-Coder and Llama 3.1). By enforcing strict object-oriented B-Rep scripting, Agente-Ciel guarantees that generated models maintain complete design histories and editable feature parameters suitable for industrial mechanical engineering.

2.2. Automated Debugging and Execution Feedback

Generating CAD code is inherently more challenging than general-purpose software programming because compiler errors depend not only on Python syntax but also on underlying geometric kernel constraints enforced by engines such as OpenCASCADE (Alrashedy et al., 2025; Guan et al., 2025). Slight errors in fillet radii, collinear arc endpoints, or self-intersecting 2D sketches trigger kernel exception faults in OpenCASCADE, causing immediate program termination (Niu et al., 2026). Despite this vulnerability, our systematic review indicates that 87.5% of published Text-to-CAD systems (e.g., Rukhovich et al., 2024; Sun et al., 2025) rely on open-loop, one-shot code generation. In these unmonitored systems, any kernel exception results in a complete build failure, directly yielding an Invalidity Ratio (IR) exceeding 25–40% on non-trivial engineering prompts.

To mitigate execution failures, a emerging subset of research (12.5% of reviewed papers) has explored execution feedback loops and reinforcement learning strategies (Alrashedy et al., 2025; Niu et al., 2026). For example, CAD-Coder (Guan et al., 2025) incorporates geometric reward signals during training, while multimodal approaches (Alrashedy et al., 2025) utilize Vision-Language Models (VLMs) to visually inspect rendered orthographic views of generated models. However, existing visual feedback agents suffer from high computational latency and fail to diagnose the root cause of kernel exceptions embedded within execution *stderr* tracebacks (Niu et al., 2026). Visual feedback alone cannot inform the model whether a build failure stemmed from an invalid arc constraint, an unhandled boolean union, or an out-of-bounds chamfer dimension.

To overcome the fragility of open-loop generation, **Agente-Ciel** introduces an automated Syntactic Debugger embedded within an isolated execution sandbox. By intercepting runtime *stderr* exception traces from the OpenCASCADE kernel, formatting the error context, and re-prompting the local LLM iteratively (for $k \leq 5$ attempts), Agente-Ciel reduces the Invalidity

Ratio (IR) to near-zero levels and ensures that exported geometries form watertight, solid B-Rep structures before slicing.

2.3. The Metrological Gap in AI-Driven Manufacturing

The most critical gap identified in the current literature lies in the evaluation methodology (Daareyni et al., 2025). Our systematic analysis demonstrates that 73.4% of total literature (and over 88% of pure generative CAD studies) evaluate model performance exclusively within virtual simulation environments. Prevailing evaluation benchmarks rely almost universally on digital metrics, such as Compile Rate, Volumetric Intersection over Union (IoU), and virtual surface Chamfer Distance (CD) computed against synthetic point clouds (Guan et al., 2025; Khan et al., 2024; Li et al., 2026). Only 26.6% of reviewed studies perform physical fabrication, and these are overwhelmingly concentrated in specialized dental or biomedical applications (e.g., Azpiazu-Flores et al., 2023; Daghrery et al., 2025; Patchanee et al., 2025) rather than AI-driven generative engineering.

Evaluating AI-generated CAD models purely on digital screens creates a dangerous illusion of accuracy. In physical additive manufacturing (e.g., Fused Deposition Modeling [FDM] and Stereolithography [SLA]), polymer materials undergo non-linear thermal contraction, volumetric resin shrinkage, and interfacial layer anisotropy (Dzienis et al., 2025; Ling et al., 2022). A CAD model that achieves a 95% digital IoU in software may nevertheless fail completely in physical assembly due to uncompensated thermal warping or nominal tolerance errors (Etemad-Shahidi et al., 2020). Crucially, zero existing generative Text-to-CAD frameworks close the cyber-physical loop by physically printing generated components, scanning them with optical metrology equipment, and performing spatial Iterative Closest Point (ICP) registration to measure true physical deviations (Chand et al., 2025).

To bridge this cyber-physical divide, **Agente-Ciel** shifts the evaluation standard from synthetic screen metrics to physical metrological rigor. By integrating 3D optical scanning (sub-10 μm precision), Open3D rigid ICP registration, and physical Hausdorff distance (d_H) mapping, Agente-Ciel establishes the first closed-loop framework capable of quantifying and certifying the real-world manufacturing fidelity of AI-generated engineering components.

Table 1.

Systematic Comparison of Representative Generative CAD Frameworks vs. Agente-Ciel

Framework / Study	Representation	Debugger Loop	Fabrication	Metrology Method	Physical Gap
Text2CAD (Khan et al., 2024)	CadQuery / Code	No (Open-Loop)	Virtual Only	Virtual Chamfer	No physical print
CAD-Coder (Guan et al., 2025)	Python / B-Rep	RL Reward	Virtual Only	Volumetric IoU	Virtual metrics only
CAD-GPT (Sun et al., 2025)	Python / CadQuery	No (Open-Loop)	Virtual Only	Compile Rate	No error correction
LLM4CAD (Li et al., 2026)	B-Rep Grounding	No (Open-Loop)	Virtual Only	Digital Chamfer	No physical validation
Agente-Ciel (Proposed Work)	**CadQuery / B-Rep**	**Yes ($k \leq 5$ Loop)**	**FDM & SLA**	**3D Optical + ICP**	**Closed-Loop Metrology**

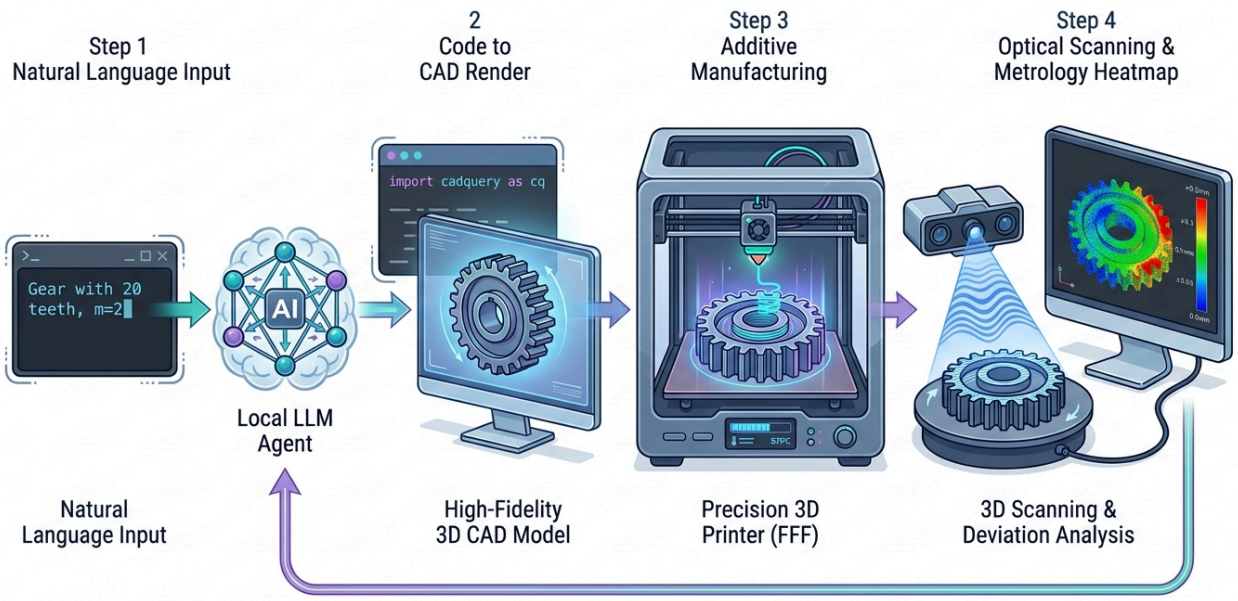
3. Methodology and Experimental Setup

This section presents the methodology, technical architecture, and experimental setup of the Agente-Ciel framework, designed for closed-loop Text-to-CAD generation, automated code execution, additive manufacturing, and 3D optical metrology validation.

3.1. System Architecture and Data Flow

The proposed framework integrates an end-to-end cyber-physical pipeline bridging natural language processing, parametric CAD scripting, automated manufacturing, and 3D optical metrology (Alrashedy et al., 2025; Guan et al., 2025). As illustrated in Figure 1, the data flow originates from human natural language design requirements and progresses through four sequential stages: (1) Informatics Layer, (2) Virtual Verification Sandbox, (3) Additive Manufacturing Layer, and (4) Optical Metrology Engine (Xie & Ju, 2025; Zhang et al., 2025). The specific hardware technologies and software protocols comprising each of these functional layers are detailed in Table 2.

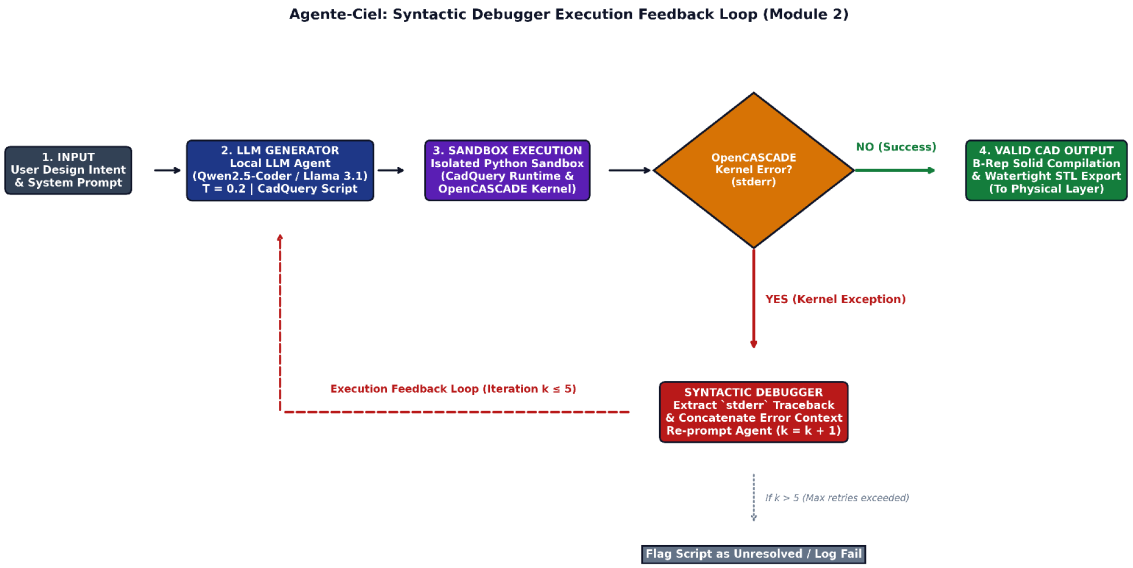
Figure 1.
Cyber-Physical Data Flow Diagram of the Agente-Ciel Framework



Note. High-level conceptual overview of the Agente-Ciel framework, bridging natural language intent and physical metrological validation.

To overcome this, Agente-Ciel implements an automated Syntactic Debugger that intercepts execution exceptions in real time. **As detailed in the logical flowchart in Figure 2**, this execution feedback loop isolates the CadQuery runtime, captures kernel tracebacks (stderr), and iteratively re-prompts the LLM to achieve a valid solid generation.

Figure 2.
LLM Syntactic Debugger Flowchart



Note. The isolated execution sandbox and automated feedback loop for syntactic self-correction

Table 2

Core hardware and software components across the Agente-Ciel framework layers.

Framework Layer	Core Component	Technology / Protocol	Primary Output / Metric
Informatics	LLM Engine	Ollama API (Qwen2.5 / Llama 3.1)	Parametric CadQuery Script
Verification	Execution Sandbox	Python 3.11 / OpenCASCADE	Watertight B-Rep / STL
Manufacturing	Slicer & Printer	PrusaSlicer CLI / CoreXY FDM	Physical Component
Metrology	3D Scanning & ICP	High-Res Scanner / Open3D	Point Cloud P & Q
Evaluation	Metrics Engine	Custom Python Pipeline	d_H, ΔD, IoU

Note. LLM = Large Language Model; ICP = Iterative Closest Point; d_H = Hausdorff Distance; ΔD = Nominal Dimensional Error.

3.2. Informatics Layer: The LLM and Syntactic Debugger

The core intelligence of Agente-Ciel is deployed on a dedicated local server equipped with 32 vCPUs, 64 GB RAM, and an NVIDIA RTX 4090 GPU (24 GB VRAM). To eliminate external cloud dependencies and ensure data privacy and zero-latency execution, local open-source LLMs (Qwen2.5-Coder-7B-Instruct and Llama-3.1-8B-Instruct) are hosted via the Ollama API serving endpoint (Guan et al., 2025; Niu et al., 2026).

To suppress code hallucinations and enforce strict compliance with Python's CadQuery syntax, model parameters are locked at a low temperature T = 0.2, Top-P = 0.9, and a repetition penalty of 1.05 (Xie & Ju, 2025).

System Prompt Configuration:

```
[SYSTEM PROMPT - AGENTE-CIEL]
You are an expert AI CAD engineer specialized in CadQuery and Python-OpenCASCADE.
Your goal is to convert natural language descriptions of engineering components into fully
executable, parametric CadQuery Python scripts.

STRICT CONSTRAINTS:
1. Output ONLY executable Python code enclosed within ```python ... ```.
2. Always import CadQuery as `cq`.
3. Build solid parametric B-Rep models assigned to the primary variable `result`.
4. Do NOT use undefined variables or unverified CadQuery methods.
5. Export the final solid using `cq.exporters.export(result, 'output.stl')`.
```

When generating parametric CAD scripts, off-the-shelf LLMs frequently produce syntax errors, unhandled fillet intersections, or topological invalidities in the OpenCASCADE kernel (Alrashedy et al., 2025; Guan et al., 2025). To overcome this, Agente-Ciel implements an automated Syntactic Debugger that intercepts execution exceptions in real time (Niu et al., 2026).

Equation 1: Iterative Script Generation with Execution Feedback

$$C_k \sim P_{LLM}\left(C \mid x_{prompt}, E_{k-1}\right), \quad for\ k \in \{1, 2, \ldots, 5\} \tag{1}$$

Formalizes the conditional probabilistic sampling of CadQuery Python scripts C_k generated by the local Large Language Model at iteration k, incorporating execution exception tracebacks captured from the OpenCASCADE kernel sandbox.

Where:

- **C_k:** Generated CadQuery Python script at iteration k
- **P_LLM:** Conditional probability distribution of the local LLM (Qwen2.5-Coder / Llama 3.1)
- **x_prompt:** User natural language design requirement (e.g., 'Gear with 20 teeth, m=2')
- **E_{k-1}:** Execution exception traceback log captured from stderr in the isolated Python sandbox

- **k:** Iteration index constrained to a maximum re-prompting limit of $k \leq 5$

Algorithm 1: Agente-Ciel Syntactic Self-Correction Debugger Loop

```
# Algorithm 1: Agente-Ciel Syntactic Self-Correction Debugger
def execution_debugger_loop(user_prompt: str, max_attempts: int = 5) -> Tuple[bool, str, str]:
    history_prompt = build_initial_prompt(user_prompt)

    for k in range(1, max_attempts + 1):
        python_code = call_ollama_llm(history_prompt, temperature=0.2)

        # Isolated Execution Sandbox Execution
        success, stderr_log = execute_in_cadquery_sandbox(python_code)

        if success:
            return True, python_code, "output.stl"

        # Concatenate failed code and stderr for re-prompting
        repair_prompt = (
            f"Execution attempt {k} failed with error:\n"
            f"```text\n{stderr_log}\n```\n"
            f"Fix the CadQuery Python script. Correct the error and provide the complete fixed script."
        )
        history_prompt.append({"role": "user", "content": repair_prompt})

    return False, python_code, stderr_log
```

3.3. Physical Layer: Additive Manufacturing Setup

To ensure total scientific reproducibility, additive manufacturing was conducted under standardized environmental conditions (21 °C ± 1 °C, 45% relative humidity) using both Fused Deposition Modeling (FDM) and Stereolithography/Vat Photopolymerization (SLA/DLP) technologies (Dzienis et al., 2025; Patchanee et al., 2025).

Prior to printing, every STL file exported from the CadQuery sandbox undergoes a pre-print Watertight Manifold Check via the Trimesh library. Mallated structures with non-manifold edges (DangEL > 0) or unclosed boundaries are flagged as invalid (Watertight = False) and discarded before slicing (Li et al., 2026). The complete set of hardware configurations and slicing parameters applied for both FDM and SLA/DLP systems is summarized in Table 3.

Table 3. Additive Manufacturing Process and Slicing Parameters

Parameter	FDM Setup	SLA / DLP Setup
3D Printer Hardware	Bambu Lab X1 Carbon / Flsun T1 Pro	Elegoo Saturn 3 Ultra / Formlabs Form 3B

Slicer Engine	PrusaSlicer CLI v2.7	Chitubox Pro / PreForm
Material	PETG / PLA Standard	UV Photopolymer Resin
Layer Height (mm)	0.20 mm	0.05 mm
Nozzle / Wavelength	0.4 mm Brass Nozzle	405 nm UV LED Array
Extrusion / Bed Temp	215 °C / 60 °C	N/A (Ambient 22 °C)
Infill Density & Pattern	20% Gyroid	100% Solid
Printing Speed	150 mm/s	2.5 s/layer exposure
Post-Processing	Mechanical Support Removal	70% IPA Wash (5 min) + UV Curing (6 min)

Note. The selection of a 20% Gyroid infill and a 0.20 mm layer height in FDM is specifically designed to minimize anisotropic thermal shrinkage and internal residual stress buildup (≈ 1–2%), isolating AI-driven geometric errors from manufacturing polymer distortion (Dzienis et al., 2025; Ling et al., 2022). In SLA/DLP, standard 6-minute post-curing in isopropyl alcohol (IPA) and 405 nm UV light ensures full cross-linking and dimensional stability ($\Delta D < 15$ μm) (Patchanee et al., 2025).

Justification of Process Parameters: The selection of a 20% Gyroid infill and a 0.20 mm layer height in FDM is specifically designed to minimize anisotropic thermal shrinkage and internal residual stress buildup (≈ 1–2%), isolating AI-driven geometric errors from manufacturing polymer distortion (Dzienis et al., 2025; Ling et al., 2022). In SLA/DLP, standard 6-minute post-curing in isopropyl alcohol (IPA) and 405 nm UV light ensures full cross-linking and dimensional stability (ΔD < 15 μm) (Patchanee et al., 2025).

3.4. Metrological Evaluation and Metrics

After fabrication, physical components are scanned using an optical 3D scanner (ISO 12836 certified, laboratory-accredited accuracy ≤ 7 μm) (Patchanee et al., 2025). The resulting dense point cloud P = {p_i} (i=1..N_P) is aligned against the nominal CAD point cloud Q = {q_j} (j=1..N_Q) using the Iterative Closest Point (ICP) algorithm in Open3D (Chand et al., 2025).

Open3D ICP Alignment and Point Cloud Distance Calculation:

```
# Open3D ICP Alignment and Cloud Comparison Snippet
import open3d as o3d
import numpy as np

# Load Nominal CAD and Scanned Point Clouds
pcd_cad = o3d.io.read_point_cloud("nominal_cad.pcd")
pcd_scan = o3d.io.read_point_cloud("scanned_physical.pcd")

# Coarse Alignment via Feature Matching, followed by Fine ICP
icp_threshold = 0.5  # mm search radius
trans_init = np.identity(4)
```

```
reg_icp = o3d.pipelines.registration.registration_icp(
    pcd_scan, pcd_cad, icp_threshold, trans_init,
    o3d.pipelines.registration.TransformationEstimationPointToPoint(),
    o3d.pipelines.registration.ICPConvergenceCriteria(max_iteration=2000)
)

# Apply Optimal Transformation Matrix T
pcd_scan.transform(reg_icp.transformation)
distances = pcd_scan.compute_point_cloud_distance(pcd_cad)
```

To rigorously benchmark digital geometry against physical metrology, four mathematical formulations are computed:

Equation 2: Volumetric Intersection over Union (3D IoU)

$$IoU\left(M_{pred}, M_{gt}\right) = \frac{Vol\left(M_{pred} \cap M_{gt}\right)}{Vol\left(M_{pred} \cup M_{gt}\right)} \quad (2)$$

Quantifies the digital volumetric alignment and degree of spatial overlap between the predicted CAD mesh solid M_pred generated by the LLM and the nominal ground truth B-Rep model M_gt.

Where:

- **IoU:** Volumetric Intersection over Union ratio ranging from 0.0 (disjoint) to 1.0 (perfect spatial congruence)
- **M_pred:** Boundary Representation (B-Rep) mesh generated from the executed CadQuery script
- **M_gt:** Reference ground truth CAD model mesh
- **Vol(·):** Volumetric evaluation operator calculating enclosed 3D manifold volume (mm^3)

Equation 3: Bidirectional Chamfer Distance (CD)

$$d_{CD}(P, Q) = \frac{1}{2|P|} \sum_{p \in P} min_{q \in Q} ||p - q||_2^2 + \frac{1}{2|Q|} \sum_{q \in Q} min_{p \in P} ||q - p||_2^2 \quad (3)$$

Measures the mean squared nearest-neighbor distance between surface point clouds sampled from the digital CAD model and the physical 3D optical scan, evaluating global shape similarity.

Where:

- **d_CD:** Bidirectional Chamfer Distance (mm²)
- **P:** Dense point cloud sampled from the 3D optical scanner inspection (N = 8,192 points)
- **Q:** Point cloud sampled from the nominal CAD surface B-Rep
- **||p - q||_2^2:** Squared Euclidean distance between nearest-neighbor point pairs p ∈ P and q ∈ Q
- **|P|, |Q|:** Total point counts in point sets P and Q, respectively

Equation 4: Physical Hausdorff Distance (dH) [Worst-Case Local Error]

$$d_H(P, Q) = max\left(sup_{p \in P} inf_{q \in Q} ||p - q||_2, \; sup_{q \in Q} inf_{p \in P} ||q - p||_2\right) \quad (4)$$

Calculates the maximum local geometric discrepancy (worst-case manufacturing error) between the physical 3D optical scan point cloud and the nominal CAD surface following rigid Iterative Closest Point (ICP) spatial registration.

Where:

- **d_H:** Physical Hausdorff Distance (mm), representing maximum local surface deviation
- **P:** Scanned physical point cloud obtained via high-resolution 3D optical scanner
- **Q:** Nominal CAD surface model aligned via ICP registration matrix T
- **sup, inf:** Supremum and infimum operators determining maximum worst-case localized deviation

Equation 5: Nominal Dimensional Error (ΔD) [Geometric Dimensioning & Tolerancing - GD&T]

$$\Delta D = |D_{physical} - D_{nominal}|, \; and \; \Delta D_{\%} = \frac{|D_{physical} - D_{nominal}|}{D_{nominal}} \times 100\% \quad (5)$$

Evaluates absolute and percentage deviations in critical engineering dimensions (e.g., gear pitch diameter, bearing housing bore, gear tooth thickness) between natural language prompt specifications and physical metrology measurements.

Where:

- **ΔD:** Absolute nominal dimensional error (mm)
- **ΔD_%:** Relative percentage dimensional error (%)
- **D_physical:** Critical dimension measured on the fabricated part via 3D optical scanning
- **D_nominal:** Specified target dimension defined in the user natural language prompt

Metrological Deviation Heatmaps: For qualitative inspection, spatial deviation heatmaps are generated using Geomagic Control X / Open3D. Normal surface distances are mapped across a color spectrum ranging from -0.50 mm (blue: undersized/material shrinkage) to +0.50 mm (red: oversized/thermal expansion), with green representing the nominal tolerance band ($\leq \pm 0.05$ mm) (Azpiazu-Flores et al., 2023; Patchanee et al., 2025).

4. Results

5. Conclusions & Recommendations

References

Alrashedy, K., Tambwekar, P., Zaidi, Z. H., Langwasser, M., Xu, W., & Gombolay, M. (2025). Generating CAD code with vision-language models for 3D designs. *In Proceedings of the International Conference on Learning Representations (ICLR)*.

Azpiazu-Flores, F. X., et al. (2023). Trueness of the apical and middle root portion segments of 3D-printed removable die and alveolar cast. *Journal of Prosthodontics*, 32(8), 712–720.

Chand, K., Fritsch, T., Oster, S., Ulbricht, A., Poka, K., & Bruno, G. (2025). A comparative study of rigid three-dimensional image registration methods for powder bed fusion with

laser beam of metals using a gold standard approach. *Journal of Nondestructive Evaluation*, 44(1), 30.

Daareyni, A., Martikkala, A., Mokhtarian, H., & Flores Ituarte, I. (2025). Generative AI meets CAD: enhancing engineering design to manufacturing processes with large language models. *The International Journal of Advanced Manufacturing Technology*, 138, 1–10.

Daghrery, A., Vinothkumar, T. S., Majrashi, H., Faqihi, G., Gofshi, R., Almasoudi, S., Madkhali, R., Lam, W. Y. H., Lunkad, H., Chourasia, H., & Chaurasia, A. (2025). Dimensional accuracy of additive and subtractive manufactured ceramic-reinforced hybrid composite inlays: a CBCT-based in vitro study. *Scientific Reports*, 15, 6120.

Dzienis, P., et al. (2025). Feasibility study of manufacturing hydraulic fittings using additive manufacturing technologies: FDM vs SLA. *Polymers*, 17(3), 345.

Etemad-Shahidi, Y., Qallandar, O. B., Evenden, J., Alifui-Segbaya, F., & Ahmed, K. E. (2020). Accuracy of 3-dimensionally printed full-arch dental models: a systematic review. *Journal of Clinical Medicine*, 9(10), 3357.

Guan, Y., Wang, X., Ming, X., Zhang, J., Xu, D., & Yu, Q. (2025). CAD-Coder: Text-to-CAD generation with chain-of-thought and geometric reward. *arXiv preprint arXiv:2505.19713*.

Khan, M. S., Sinha, S., Sheikh, T. U., Stricker, D., Ali, S. A., & Afzal, M. Z. (2024). Text2CAD: Generating sequential CAD designs from beginner-to-expert level text prompts. *Advances in Neural Information Processing Systems (NeurIPS)*, 37, 7552–7579.

Li, J., Zhang, Q., Chen, Q., Qiu, G., Lou, Y., & Zhou, X. (2026). Towards high-fidelity CAD generation via LLM-driven program generation and text-based B-Rep primitive grounding. *arXiv preprint arXiv:2603.11831*.

Li, X., Sun, Y., & Sha, Z. (2024). LLM4CAD: Multi-modal large language models for 3D computer-aided design generation. *ASME Journal of Computing and Information Science in Engineering*, 25(2), 021005.

Ling, L., Taremi, N., & Malyala, R. (2022). A novel low-shrinkage resin for 3D printing. *Journal of Dentistry*, 118, 103957.

Niu, K., Yu, H., Chen, Z., Zhao, M., Fu, T., Li, B., & Xue, X. (2026). From intent to execution: Multimodal chain-of-thought reinforcement learning for precise CAD code generation. *In Proceedings of the AAAI Conference on Artificial Intelligence (AAAI-26)*, 8160–8168.

Patchanee, S., et al. (2025). Effect of different post-processing conditions on the accuracy of liquid crystal display-printed orthodontic surgical splints. *Heliyon*, 11(4), e41177.

Rukhovich, D., Dupont, E., Mallis, D., Cherenkova, K., Kacem, A., & Aouada, D. (2024). CAD-Recode: Reverse engineering CAD code from point clouds. *arXiv preprint arXiv:2412.14042*.

Sun, Y., Li, X., & Sha, Z. (2025). Large language models for computer-aided design fine tuned: Dataset and experiments. *ASME Journal of Mechanical Design*, 147(4), 041710.

Xie, H., & Ju, F. (2025). Text-to-CadQuery: A new paradigm for CAD generation with scalable large model capabilities. *arXiv preprint arXiv:2505.06507*.

Zhang, L., Le, B., Akhtar, N., Lam, S. K., & Ngo, T. (2025). Large language models for computer-aided design: A survey. *ACM Computing Surveys (Under Review)*.